Η κλάση Keyboard διαθέτει μια απλή μέθοδο read που για απλότητα επιστρέφει μια συμβολοσειρά. Μπορείτε να αναπτύξετε την read ώστε πράγματι να λαμβάνει μια είσοδο από το πληκτρολόγιο και να την επιστρέφει. Όπως και να έχει, δεν επηρεάζονται οι έννοιες που παρουσιάζουμε εδώ.

```
public class Memory {

     private String content=null;

     public String getContent() {
          return content;
     }

     public void setContent(String content) {
          this.content = content;
     }

     public boolean isEmpty() {
          return content==null;
     }
}
```

Η κλάση Memory έχει τρεις λειτουργίες. Μπορεί να αποθηκεύσει μια τιμή τύπου String, να επιστρέψει την αποθηκευμένη τιμή και να ελέγξει αν υπάρχει ή όχι αποθηκευμένη τιμή.
Η κλάση Monitor μπορεί απλώς να εμφανίσει μια τιμή String.

```
public class Monitor {

     public void display(String data) {
          System.out.println(data);
     }
}
```

Τέλος, στον κώδικα 13.8 παρουσιάζεται η κλάση Computer:

```
public class Computer extends ElectricDevice {

    private final Keyboard keyboard = new Keyboard();
    private final Memory memory = new Memory();
    private final Monitor monitor = new Monitor();

    void keyboardToMem() {
        if (!isPowerOn()) {
            return;
        }
        String data = keyboard.read();
        memory.setContent(data);
    }

    void memToMonitor() {
        if (!isPowerOn() || memory.isEmpty()) {
            return;
        }
        monitor.display(memory.getContent());
    }

    void clearMem() {
```

```
            memory.setContent(null);
        }

        public void operate() {
            clearMem();
            keyboardToMem();
            memToMonitor();
        }

        public static void main(String[] args) {
            Computer alfaX = new Computer();
            alfaX.setPowerOn(true);
            alfaX.operate();
            alfaX.setPowerOn(false);
        }
    }
```

Κώδικας 13.8 *Η κλάση Computer*

Όπως φαίνεται στον κώδικα 13.8, η κλάση Computer κληρονομεί την κλάση ElectricDevice, ενώ διαθέτει ιδιωτικά μέλη των κλάσεων Memory, Keyboard και Monitor. Έτσι, ο Computer alfaX που δημιουργείται στην main, μπορεί να συνδεθεί και να αποσυνδεθεί από το ρεύμα με τη βοήθεια της setPowerOn που κληρονομεί από τις ηλεκτρικές συσκευές, να μεταφέρει την είσοδο του πληκτρολογίου του στη μνήμη του, να στέλνει το περιεχόμενο της μνήμης του στην οθόνη του, να αρχικοποιεί το περιεχόμενο της μνήμης του.

Πέραν των δύο βασικών σχέσεων, της σύνθεσης και κληρονομικότητας που μας ενδιαφέρουν στο πλαίσιο αυτού του βιβλίου, έχουν θεσπιστεί και άλλοι τύποι σχέσεων ώστε να υπάρχει η δυνατότητα οι σχέσεις μεταξύ των κλάσεων να προσδιορίζονται με μεγαλύτερη λεπτομέρεια [1].

13.1.4 Η κλάση Object

Η κλάση Object είναι η γονική κλάση κάθε κλάσης της Java. Τόσο οι κλάσεις της βιβλιοθήκης, όσο και οι οριζόμενες από τον χρήστη κλάσεις κληρονομούν αυτομάτως, χωρίς δηλαδή να το δηλώσουμε σαφώς με τη λέξη-κλειδί extends, την κλάση Object. Το ίδιο ισχύει και για τους πίνακες της Java οι οποίοι στην ουσία συνιστούν κλάσεις για τις οποίες προσφέρεται ένα διαφορετικό συντακτικό δημιουργίας και διαχείρισης.

Η κλάση Object περιλαμβάνει καταρχάς τις γνώριμές μας συναρτήσεις toString, equals και hashCode. Η συνάρτηση toString επιστρέφει μια συμβολοσειρά που αποτελείται από το όνομα του πακέτου ακολουθούμενο από το όνομα της κλάσης συνδεδεμένο με τον χαρακτήρα '@' και τον κωδικό κατακερματισμού του αντικειμένου εκφρασμένο σε μη προσημασμένη δεκαεξαδική μορφή. Αν τυπώσετε ένα αντικείμενο της κλάσης, θα λάβετε κάτι σαν java.lang.Object@4f3f5b2. Αν δημιουργήσετε μια κλάση, η συνάρτηση toString είναι αυτομάτως διαθέσιμη, καθώς κληρονομείται από την κλάση Object. Το ίδιο ισχύει για όλες τις συναρτήσεις της Object. Όποτε δεν μας διευκολύνει η toString, πράγμα που ισχύει συνήθως, μπορούμε να την επανορίσουμε στο επίπεδο της κλάσης που δημιουργήσαμε.

Η συνάρτηση equals της Object, για 2 μεταβλητές, o1 και o2 τύπου Object, θα επιστρέψει true μόνον εφόσον o1==o2, δηλαδή μόνον εφόσον οι δύο μεταβλητές αναφέρονται στην ίδια θέση μνήμης ή αλλιώς στο ίδιο αντικείμενο. Αυτή η σύγκριση τις περισσότερες φορές δεν μας εξυπηρετεί, καθώς αντικείμενα με το ίδιο περιεχόμενο μπορεί να έχουν κατανεμηθεί σε διαφορετικές θέσεις μνήμης. Έτσι, αποτελεί σχεδόν κανόνα ο επανορισμός της equals στις κλάσεις που δημιουργούμε. Εξαίρεση σε αυτόν τον κανόνα αποτελούν κλάσεις που εγγυώνται πως μόνο ένα αντικείμενο με συγκεκριμένο περιεχόμενο μπορεί να δημιουργηθεί κατά την εκτέλεση της εφαρμογής. Χαρακτηριστικό παράδειγμα είναι οι απαριθμήσιμοι τύποι οι οποίοι διατηρούν τον δημιουργό τους ιδιωτικό, αποτρέποντας έτσι τον χρήστη τους να δημιουργεί διαφορετικά στιγμιότυπα του ίδιου αντικειμένου. Ας θεωρήσουμε ένα απαριθμήσιμο τύπο για τις ημέρες της εβδομάδας με αντικείμενα SUNDAY έως SATURDAY. Υπάρχει εγγύηση πως μόνο ένα αντικείμενο, π.χ. SUNDAY έχει δημιουργηθεί. Μπορεί να έχουμε πολλές αναφορές σε αυτό το αντικείμενο. Το ίδιο το αντικείμενο όμως υπάρχει μια μόνο φορά στη μνήμη της εφαρμογής μας. Σε αυτήν την περίπτωση δεν υπάρχει ανάγκη επανορισμού της equals. Μια διαφορετική περίπτωση είναι κλάσεις που χρησιμοποιούνται ως πρόχειρες μεταβλητές για διαμόρφωση δεδομένων, π.χ. η κλάση StringBuilder. Τα αντικείμενα StringBuilder χρησιμοποιούνται για τη διαμόρφωση αντικειμένων String. Μπορεί σε κάποια φάση κατά τη διαμόρφωσή τους δύο αντικείμενα StringBuilder να

παρουσιάζουν ίδιο περιεχόμενο, αργότερα όμως να καταλήξουν σε διαφορετικά περιεχόμενα. Επομένως, θεωρήθηκε πως δεν έχει νόημα η σύγκριση του περιεχομένου δύο αντικειμένων StringBuilder. Εξάλλου, αν κάπου χρειαστείτε τέτοια σύγκριση, μπορείτε πάντα να καλέσετε sb1.toString().equals(sb2.toString()), όπου sb1 και sb2 αντικείμενα StringBuilder.

Άλλες συναρτήσεις της Object που θα πρέπει να αναφέρουμε εδώ είναι οι:

```
protected Object clone()
```

Επιστρέφει ένα αντίγραφο του αντικειμένου που την καλεί. Η συνάρτηση έχει προστατευμένη πρόσβαση και επομένως είναι διαθέσιμη σε κάθε κλάση της Java, αφού όλες οι κλάσεις έχουν ως γονική την Object. Ωστόσο, για να χρησιμοποιηθεί η συνάρτηση θα πρέπει η κλάση να υλοποιεί τη διεπαφή Cloneable, διαφορετικά θα παράγει εξαίρεση. Για τις διεπαφές θα μιλήσουμε αναλυτικά στην ενότητα 14. Η περίεργη αυτή συμπεριφορά οφείλεται στο γεγονός πως η συνάρτηση ορίστηκε στην κλάση Object και εν συνεχεία έγινε αντιληπτό πως υπάρχει η ανάγκη για κάποιες κλάσεις να αποτρέπεται η δημιουργία αντιγράφων των αντικειμένων τους.

```
public final Class<?> getClass()
```

Στην ουσία επιστρέφει την κλάση στην οποία ανήκει το αντικείμενο που την καλεί. Όπως φαίνεται από τη διεπαφή της, η τιμή επιστροφής της είναι μια γενικευμένη κλάση. Έχοντας ένα τέτοιο αντικείμενο μπορούμε να προσπελάσουμε χαρακτηριστικά της κλάσης που ανήκει αυτό το αντικείμενο, π.χ.

```
ElectricDevice eD = new ElectricDevice();
System.out.println(eD.getClass().getName());
```

13.1.5 Η σειρά αρχικοποίησης

Τα μέλη της κλάσης σε αντίθεση με τις τοπικές μεταβλητές αρχικοποιούνται αυτόματα. Επιπλέον, μπορεί ο ιδιοκτήτης της κλάσης να τα αρχικοποιήσει σαφώς εκεί που τα δηλώνει. Πέραν αυτού είναι δυνατόν σε μία κλάση να προστεθεί ένα ή περισσότερα μπλοκ αρχικοποίησης (Initialization block). Τα μπλοκ αρχικοποίησης δεν έχουν και ιδιαίτερη χρησιμότητα και θα τα συναντήσετε πολύ σπάνια σε κώδικες Java. Ωστόσο, αν χρησιμοποιηθούν θα πρέπει να διαφοροποιούνται αυτά για τις στατικές μεταβλητές από τα μπλοκ για τις μεταβλητές στιγμιότυπου. Ο κώδικας που ακολουθεί επιδεικνύει αυτές τις πιθανές αρχικοποιήσεις:

```
import java.util.ArrayList;

public class Initialization {

     ArrayList<String> store = new ArrayList<>();
     String id, otherData;
     static boolean check;

     static { //Initialization Block for static members
          check = true;
     }

     { //Initialization Block
          otherData = "OtherData";
     }

     public Initialization(String id) {
          this.id = id;
     }
}
```

Πρώτα από όλα αρχικοποιείται το στατικό μέλος check. Το μέλος store αρχικοποιείται σαφώς κατά τη δήλωσή του. Στη συνέχεια τρέχει το μπλοκ αρχικοποίησης όπου αρχικοποιείται το μέλος otherData. Τέλος το id αρχικοποιείται στον δημιουργό της κλάσης. Τι ισχύει όμως όταν η κλάση είναι παράγωγη; Γενικά, η προτεραιότητα αρχικοποίησης έχει ως εξής:

1. Στατικά πεδία και στατικά μπλοκ προγόνων.
2. Στατικά πεδία και στατικό μπλοκ της τρέχουσας κλάσης.
3. Μεταβλητές στιγμιότυπου και μπλοκ αρχικοποίησης προγόνων.
4. Δημιουργός του προγόνου.
5. Μεταβλητές στιγμιότυπου και μπλοκ αρχικοποίησης της τρέχουσας κλάσης.
6. Δημιουργός της τρέχουσας τάξης.

Δύο ή περισσότερα στοιχεία με την ίδια προτεραιότητα αρχικοποιούνται σύμφωνα με τη σειρά τοποθέτησής τους στον κώδικα.

13.2 Πολυμορφισμός

Ένα προφανές πλεονέκτημα της κληρονομικότητας είναι η επαναχρησιμοποίηση κώδικα (code reusability). Οι μέθοδοι της γονικής κλάσης είναι χρήσιμες και στις παράγωγες. Με την κληρονομικότητα γίνονται αυτόματα διαθέσιμες και έτσι δεν χρειάζεται να δημιουργήσουμε αντίγραφα του ίδιου κώδικα.
Το μεγαλύτερο όμως πλεονέκτημα της κληρονομικότητας είναι πως υποστηρίζει τον πολυμορφισμό. Ο πολυμορφισμός βασίζεται στη δυνατότητα να εκχωρήσουμε σε μια μεταβλητή αναφοράς τύπου μιας γονικής κλάσης, ένα στιγμιότυπο παράγωγης. Για παράδειγμα, η εκχώρηση

```
ElectricDevice betaX = new Computer();
```

είναι απολύτως ορθή. Εφόσον ένας ηλεκτρονικός υπολογιστής είναι μια ηλεκτρική συσκευή, γιατί να μην μπορούμε να τον διαχειριστούμε σαν ηλεκτρική συσκευή; Από τη στιγμή όμως που "βλέπουμε" έναν υπολογιστή απλώς σαν ηλεκτρική συσκευή, είναι διαθέσιμες μόνο οι λειτουργίες που αυτός κληρονομεί από την κλάση ElectricDevice. Επομένως, λαμβάνοντας υπόψη τη δήλωση του betaX, η κλήση betaX.operate() θα παράγει λάθος μεταγλώττισης. Πράγματι, όταν ο μεταγλωττιστής θα συναντήσει αυτήν την κλήση, θα ελέγξει καταρχάς τι τύπου είναι η μεταβλητή betaX και θα δει πως είναι τύπου ElectricDevice. Στη συνέχεια θα ψάξει στην κλάση ElectricDevice να βρει μέθοδο με την ταυτότητα της operate. Εφόσον δεν θα την βρει, θα παράξει λάθος μεταγλώττισης.

Ωστόσο, εφόσον ο προγραμματιστής γνωρίζει πως στη μεταβλητή betaX τύπου ElectricDevice έχει εκχωρηθεί πραγματικό αντικείμενο τύπου Computer, μπορεί να προσπελάσει τις μεθόδους της κλάσης Computer με απλή μετατροπή τύπου, ((Computer)betaX).operate().

Δεν μπορούμε λοιπόν να καλέσουμε χωρίς μετατροπή τύπου τις μεθόδους της Computer από το στιγμιότυπο betaX. Μπορούμε όμως να καλέσουμε τις μεθόδους της γονικής κλάσης ElectricDevice, δηλαδή η κλήση betaX.setPowerOn(true) είναι απολύτως ορθή. Αυτή η δυνατότητα είναι πολύτιμη όπως θα διαπιστώσετε αμέσως παρακάτω.

Ας υποθέσουμε πως έχουμε και άλλους τύπους ηλεκτρικών συσκευών. Πιο συγκεκριμένα, στην εφαρμογή μας έχουμε μια κλάση που αναπαριστά τα πλυντήρια (WashingMachine), μια κλάση που αναπαριστά τους ηλεκτρικούς φούρνους (Oven) και μία κλάση που αναπαριστά τα κλιματιστικά (AirCondition). Όλες αυτές είναι ηλεκτρικές συσκευές. Στον κώδικα 13.9 δίνουμε τον ορισμό αυτών των κλάσεων:

```
public class AirCondition extends ElectricDevice {

    public void operate() {
        System.out.println("Το air condition δροσίζει τον χώρο");
    }
}

public class Oven extends ElectricDevice {

    public void operate() {
```

```
            System.out.println("Ο φούρνος ψήνει το φαγητό");
        }
    }

    public class WashingMachine extends ElectricDevice {

        public void operate() {
            System.out.println("Το πλυντηριο πλένει τα ρούχα");
        }
    }
```

Κώδικας 13.9 *Τρεις κλάσεις παράγωγες της ElectricDevice*

Ας υποθέσουμε τώρα πως η εφαρμογή μας έχει να κάνει με το έξυπνο σπίτι στο οποίο θέλουμε να έχουμε έναν τρόπο να ενεργοποιούμε, να απενεργοποιούμε και να ελέγχουμε ταυτόχρονα όλες τις διαθέσιμες ηλεκτρικές συσκευές. Πώς μπορούμε να το πετύχουμε; Παρουσιάζουμε τον κατάλληλο κώδικα και στη συνέχεια τον επεξηγούμε.

```
    public class SmartHome {

        public static void switchOn(ElectricDevice[] homeElectrics) {
            for (ElectricDevice eD : homeElectrics) {
                eD.setPowerOn(true);
            }
        }

        public static void switchOff(ElectricDevice[] homeElectrics) {
            for (ElectricDevice eD : homeElectrics) {
                eD.setPowerOn(false);
            }
        }

        public static boolean checkDevices(ElectricDevice[] homeElectrics) {
            for (ElectricDevice eD : homeElectrics) {
                if (!eD.isPowerOn()) {
                    return false;
                }
            }
            return true;
        }

        public static void main(String[] args) {
            ElectricDevice[] homeElectrics = new ElectricDevice[4];
            homeElectrics[0] = new WashingMachine();
            homeElectrics[1] = new AirCondition();
            homeElectrics[2] = new Oven();
            homeElectrics[3] = new Computer();
            switchOn(homeElectrics);
            System.out.println(checkDevices(homeElectrics));
            switchOff(homeElectrics);
            System.out.println(checkDevices(homeElectrics));
        }
    }
```

Κώδικας 13.10 *Εφαρμογή 'Το έξυπνο σπίτι'.*

Ας συζητήσουμε τον κώδικα 13.10 ξεκινώντας από την main. Στην main έχουμε ορίσει έναν πίνακα για αντικείμενα ElectricDevice. Θα μπορούσαμε να έχουμε χρησιμοποιήσει κάποια άλλη κλάση, π.χ. ArrayList<ElectricDevice>. Η επιλογή αυτή όμως δεν σχετίζεται με τις έννοιες που παρουσιάζουμε εδώ. Σε κάθε θέση του, ο πίνακας homeElectrics μπορεί να φιλοξενήσει μια μεταβλητή αναφοράς τύπου ElectricDevice και αφού είπαμε πως σε τέτοια μεταβλητή μπορούμε να εκχωρήσουμε αντικείμενα παράγωγων κλάσεων,

εκχωρούμε τις συσκευές του έξυπνου σπιτιού. Επομένως, μπορούμε τώρα από τις αναφορές που έχουμε στον πίνακα να καλέσουμε μεθόδους της ElectricDevice. Αυτό ακριβώς κάνουμε στις συναρτήσεις switchOn, switchOff και checkDevices. Ο κώδικάς μας είναι πολυμορφικός. Η συνάρτηση switchOn για παράδειγμα επενεργεί σε μεταβλητές διαφορετικών τύπων, ενώ ταυτόχρονα διατηρούμε τον απαραίτητο έλεγχο τύπου. Ο πίνακας homeElectrics δέχεται μόνο αντικείμενα κάθε ένα εκ των οποίων είναι μια ηλεκτρική συσκευή. Ωστόσο, τα αντικείμενα αυτά εξειδικεύονται περαιτέρω ως προς τον τύπο τους. Για όλα όμως υπάρχει η εγγύηση ότι μπορούν να καλέσουν τις συναρτήσεις που έχουν οριστεί στη γονική κλάση.

Ας υποθέσουμε τώρα πως κάποιες μέθοδοι της ElectricDevice έχουν επανορισθεί στις παράγωγες. Έστω ότι τα αντικείμενα της κλάσης Computer πριν συνδεθούν στο ρεύμα, πρέπει να συνδεθούν σε κατάλληλο μετασχηματιστή. Αυτό μπορούμε να το πετύχουμε εύκολα προσθέτοντας τον κώδικα 13.11 στην κλάση Computer.

```
private void connectToTransformer() {
    System.out.println("connected");
}

@Override
public void setPowerOn(boolean powerOn) { //Κώδικας 13.11
    if (powerOn) {
        connectToTransformer();
    }
    super.setPowerOn(powerOn);
}
```

Κώδικας 13.11 *Προσθήκη λειτουργίας σύνδεσης σε μετασχηματιστή και επανορισμός της setPowerOn στην κλάση Computer.*

Το ερώτημα που τίθεται εδώ είναι ποια setPowerOn θα καλέσει ο κώδικας 13.10. Αυτήν που ορίσθηκε στην ElectricDevice ή αυτήν που ορίσαμε στην κλάση Computer. Είναι γεγονός πως πριν μετατρέψουμε την κλάση Computer, ο κώδικας καλεί την setPowerOn της ElectricDevice. Μετά όμως τη μετατροπή της κλάσης Computer χωρίς καμία επιπλέον μετατροπή, ο κώδικας 13.10 θα καλέσει την setPowerOn που επανορίστηκε στην κλάση Computer. Ισχύει λοιπόν ο γενικός κανόνας πως από μια αναφορά τύπου γονικής καλούνται μόνο συναρτήσεις για τις οποίες υπάρχει ορισμός στη γονική, ωστόσο καλείται η υλοποίηση της κλάσης που αντιστοιχεί στο πραγματικό αντικείμενο. Αν στην παράγωγη έχει επανοριστεί μια μέθοδος καλείται αλλιώς η κληρονομούμενη. Το χαρακτηριστικό αυτό ονομάζεται δυναμική δέσμευση (dynamic binding), καθώς η ακριβής ταυτότητα της μεθόδου που θα κληθεί υπολογίζεται κατά τον χρόνο εκτέλεσης και όχι κατά τον χρόνο μεταγλώττισης.

Αυτό το χαρακτηριστικό πολλαπλασιάζει την αξία της κληρονομικότητας και του πολυμορφισμού, γεγονός που θα έχουμε πολλές ευκαιρίες να διαπιστώσουμε στη συνέχεια.

13.2.1 Ο τελεστής instanceof

Όπως είδαμε σε μια μεταβλητή αναφοράς τύπου γονικής είναι εφικτό να εκχωρήσουμε αναφορά τύπου παράγωγης, δηλαδή η εκχώρηση

```
ElectricDevice eD = new WashingMachine();
```

είναι απολύτως σωστή. Αυτό σημαίνει πως μέσα στο πρόγραμμά μας είναι δυνατόν να έχουμε μεταβλητές τύπου γονικής που όμως αναφέρονται σε πραγματικά στιγμιότυπα παράγωγης, όπως ακριβώς η μεταβλητή eD. Είναι επίσης πιθανό, η παράγωγη κλάση να έχει ορίσει επιπλέον δικές της μεθόδους που δεν υφίστανται στη γονική. Για παράδειγμα, ας θεωρήσουμε πως η κλάση WashingMachine περιλαμβάνει μια adjustWaterTemperature, όπως δείχνει ο κώδικας 12.13.

```
public class WashingMachine extends ElectricDevice {

    double temperature = 20;

    public void adjustWaterTemperature(double temperature) {
```

```
            this.temperature = temperature;
        }

        @Override
        public void operate() {
            System.out.println("Το πλυντηριο πλένει τα ρούχα στους
    "+temperature+" βαθμούς");
        }
    }
```

Κώδικας 13.12 *Η κλάση WashingMachine εφοδιασμένη με την μέθοδο adjustWaterTemperature.*

Η μέθοδος adjustWaterTemperature δεν μπορεί να κληθεί από το αντικείμενο eD του οποίου ο τύπος είναι ElectricDevice. Ο μόνος τρόπος για να κληθεί είναι η μετατροπή τύπου του στιγμιότυπου eD από ElectricDevice σε WashingMachine (downcasting). Είναι όμως ασφαλής μια τέτοια μετατροπή; Κάτω από ποιες συνθήκες μπορούμε να είμαστε σίγουροι πως μια μεταβλητή τύπου ElectricDevices αναφέρεται σε πραγματικό αντικείμενο τύπου WashingMachine; Την απάντηση μας δίνει ο τελεστής instanceof με τον οποίο ελέγχουμε τον τύπο του πραγματικού αντικειμένου μιας μεταβλητής αναφοράς. Ο κώδικας 13.13 επιδεικνύει τη χρήση του τελεστή instanceof.

```
public class InstanceOfDemo {

    public static void main(String[] args) {

        ElectricDevice eD = new WashingMachine();
        eD.setPowerOn(true);
        if (eD instanceof WashingMachine) {
            ((WashingMachine) eD).adjustWaterTemperature(30);
        }
        eD.operate();
        System.out.println(eD instanceof ElectricDevice);
    }
}
```

Κώδικας 13.13 *Χρήση του τελεστή instanceof.*

Προσέξτε πως η έκφραση στην τρίτη γραμμή της main αλλά και η έξοδος της τελευταίας γραμμής της main είναι true. Πράγματι, η μεταβλητή eD είναι ένα πλυντήριο αλλά είναι και μια ηλεκτρική συσκευή. Ο τελεστής instanceof θα επιστρέψει true για κάθε κλάση με την οποία η εξεταζόμενη μεταβλητή σχετίζεται με τη σχέση ‘είναι ένα’.

13.3 Αφηρημένες κλάσεις

Θα παρατηρήσατε πως όλες οι κλάσεις ηλεκτρικών συσκευών που έχουμε ορίσει διαθέτουν μια μέθοδο operate. Πρόκειται για τη μέθοδο που τις θέτει σε λειτουργία. Στην εφαρμογή έξυπνο σπίτι θα θέλαμε να έχουμε έναν τρόπο να θέσουμε σε λειτουργία αυτομάτως όλες τις ηλεκτρικές συσκευές ενός σπιτιού. Δυστυχώς, έτσι όπως είναι ο κώδικάς μας κάτι τέτοιο δεν είναι εφικτό. Αν το επιχειρήσουμε, όπως εξηγήσαμε προηγουμένως, ο μεταγλωττιστής θα διαμαρτυρηθεί, καθώς δεν θα βρει μέθοδο operate στη γονική κλάση ElectricDevice. Βέβαια αυτό μπορούμε να το λύσουμε αν προσθέσουμε μια μέθοδο operate στην κλάση ElectricDevice. Τι να κάνει όμως μια τέτοια μέθοδος στην ElectricDevice; Η λειτουργία της κλάσης WashingMachine είναι να πλένει τα ρούχα, της Oven να ψήνει το φαγητό, της AirCondition να δροσίζει τον χώρο. Ποια είναι όμως η λειτουργία της ElectricDevice; Παρότι κάθε ηλεκτρική συσκευή επιτελεί κάποια λειτουργία, στο επίπεδο της ElectricDevice δεν μπορεί να καθοριστεί το περιεχόμενό της. Υπάρχει αλλά είναι ακαθόριστη. Με άλλα λόγια είναι αφηρημένη.

Πράγματι, το μοντέλο του αντικειμενοστρεφούς προγραμματισμού μας παρέχει τη δυνατότητα δήλωσης αφηρημένων (abstract) μεθόδων. Μία αφηρημένη μέθοδος, συμπεριλαμβάνει στη δήλωσή της τη λέξη-κλειδί abstract και δεν ακολουθείται από μπλοκ κώδικα, αντίθετα τερματίζεται από τον χαρακτήρα ‘;’. Προσθέστε στην κλάση ElectricDevive την αφηρημένη μέθοδο operate.

```
public abstract void operate();
```

Μία κλάση που περιλαμβάνει μία ή περισσότερες αφηρημένες μεθόδους, πρέπει και η ίδια να οριστεί ως αφηρημένη. Μετατρέψτε την επικεφαλίδα της κλάσης ElectricDevice ως ακολούθως:

```
public abstract class ElectricDevice {…}
```

Αντίθετα, μια κλάση μπορεί να οριστεί να είναι αφηρημένη ακόμη και αν καμία μέθοδός της δεν είναι αφηρημένη. Ποιο είναι όμως ακριβώς το νόημα της λέξης-κλειδί abstract;

Ο μεταγλωττιστής δεν επιτρέπει τη δημιουργία αντικειμένων μιας abstract κλάσης. Αν υποθέσουμε πως είχαμε τη δυνατότητα να δημιουργήσουμε ένα αντικείμενο της αφηρημένης έκδοσης της ElectricDevice, τι θα συνέβαινε αν το αντικείμενο καλούσε την abstract μέθοδο operate; Είναι προφανές πως η εφαρμογή μας θα οδηγείτο σε αδιέξοδο με την κλήση μιας μεθόδου για την οποία δεν έχει οριστεί σώμα.

Οι παράγωγες κλάσεις της ElectricDevice κληρονομούν και τις αφηρημένες μεθόδους. Αν τις ορίσουν, τότε είναι σαφείς (concrete) κλάσεις, δηλαδή κλάσεις που παρέχουν ορισμό όλων των μεθόδων τους και για τις οποίες μπορούμε να δημιουργήσουμε αντικείμενα. Αν δεν οριστούν οι κληρονομούμενες αφηρημένες μέθοδοι στην παράγωγη κλάση, τότε παραμένει και αυτή αφηρημένη.

Οι αφηρημένες κλάσεις και μέθοδοι ενισχύουν την ισχύ του πολυμορφισμού. Έχοντας ορίσει την αφηρημένη μέθοδο operate στην ElectricDevice εγγυόμαστε στην ουσία πως κάθε απόγονός της διαθέτει αυτήν τη μέθοδο. Επομένως, κάθε αντικείμενο που μπορεί να δημιουργηθεί τύπου κλάσης απογόνου της ElectricDevice μπορεί με ασφάλεια να καλέσει την operate.

Έτσι μετά τη μετατροπή της ElectricDevice σε abstract, η εφαρμογή έξυπνο σπίτι μπορεί να τροποποιηθεί ώστε να ενεργοποιεί αυτόματα την operate για όλες τις ηλεκτρικές συσκευές του σπιτιού. Στον κώδικα 13.14 δίνεται η νέα έκδοση της SmartHome:

```
public class SmartHome {

    public static void switchOn(ElectricDevice[] homeElectrics) {
        for (ElectricDevice eD : homeElectrics) {
            eD.setPowerOn(true);
        }
    }

    public static void switchOff(ElectricDevice[] homeElectrics) {
        for (ElectricDevice eD : homeElectrics) {
            eD.setPowerOn(false);
        }
    }

    public static boolean checkDevices(ElectricDevice[] homeElectrics) {
        for (ElectricDevice eD : homeElectrics) {
            if (!eD.isPowerOn()) {
                return false;
            }
        }
        return true;
    }

    public static void operate(ElectricDevice[] homeElectrics) {
        for (ElectricDevice eD : homeElectrics) {
            eD.operate();
        }
    }

    public static void main(String[] args) {
        ElectricDevice[] homeElectrics = new ElectricDevice[4];
        homeElectrics[0] = new WashingMachine();
        homeElectrics[1] = new AirCondition();
```

```
        homeElectrics[2] = new Oven();
        homeElectrics[3] = new Computer();
        switchOn(homeElectrics);
        operate(homeElectrics);
        switchOff(homeElectrics);
        System.out.println(checkDevices(homeElectrics));
    }
}
```

Κώδικας 13.14 *Έκδοση της SmartHome που υποστηρίζει τη λειτουργία operate για όλες τις παράγωγες της ElectricDevice.*

13.4 Λυμένες Ασκήσεις

13.4.1 Γεωμετρικά σχήματα

Να υλοποιηθεί εφαρμογή που υπολογίζει το εμβαδόν μιας σειράς γεωμετρικών σχημάτων. Πιο συγκεκριμένα, η εφαρμογή να υποστηρίζει τον υπολογισμό του εμβαδού, ορθογώνιων παραλληλόγραμμων, τετραγώνων, κύκλων και τριγώνων. Για τα ορθογώνια, γνωρίζουμε το μήκος και το πλάτος, για τους κύκλους την ακτίνα και για τα τρίγωνα τη βάση και το ύψος. Τα τετράγωνα αναπαρίστανται σαν ορθογώνια με ίδιο μήκος και πλάτος. Η εφαρμογή θα πρέπει να φορτώσει σε μία λίστα δύο γεωμετρικά σχήματα από κάθε είδος, δηλαδή σύνολο οκτώ και στη συνέχεια να εμφανίσει την ταυτότητα κάθε σχήματος συνοδευόμενη από το εμβαδόν του, όπως δείχνει το υπόδειγμα που ακολουθεί.

```
Κύκλος ακτίνας 4.5, εμβαδόν: 28.274
Κύκλος ακτίνας 6.0, εμβαδόν: 37.699
Ορθογώνιο μήκους 3.8, πλάτους 5.0, εμβαδόν: 19
Ορθογώνιο μήκους 6.0, πλάτους 2.6, εμβαδόν: 15.6
Τετράγωνο πλευράς 9.0, εμβαδόν: 81
Τετράγωνο πλευράς 3.0, εμβαδόν: 9
Τρίγωνο βάσης 4.4, ύψους=2.0, εμβαδόν: 4.4
Τρίγωνο βάσης 4.4, ύψους=2.2, εμβαδόν: 4.84
```

Το εμβαδόν των σχημάτων θα πρέπει να εμφανίζεται με ακρίβεια τριών δεκαδικών.

Λύση

Καταρχάς, προσέξτε πως σύμφωνα με την εκφώνηση, θα πρέπει οκτώ γεωμετρικά σχήματα να φορτωθούν σε ένα ArrayList. Επομένως, χρειαζόμαστε μια κλάση που αναπαριστά τα γεωμετρικά σχήματα και η οποία θα είναι γονική για τις κλάσεις των ορθογώνιων, των κύκλων και των τριγώνων. Επειδή η κοινή λειτουργία δεν είναι παρά ο υπολογισμός του εμβαδού, η κλάση μπορεί να διαθέτει μόνο μία μέθοδο. Βεβαίως, ο υπολογισμός του εμβαδού είναι εντελώς διαφορετικός για τα ορθογώνια, τους κύκλους και τα τρίγωνα. Παρότι, γνωρίζουμε πως κάθε γεωμετρικό σχήμα από αυτά που μας ενδιαφέρουν έχει εμβαδόν, το εμβαδόν δεν ορίζεται στο επίπεδο του γεωμετρικού σχήματος. Αυτή είναι ιδανική περίπτωση για να κάνουμε την μέθοδο υπολογισμού του εμβαδού αφηρημένη στην κλάση των γεωμετρικών σχημάτων. Ακολουθεί ο κώδικας της κλάσης των γεωμετρικών σχημάτων:

```
public abstract class Shape {
    public abstract double getArea();
}
```

Κώδικας 13.15 *Η αφηρημένη κλάση των γεωμετρικών σχημάτων.*

Στη συνέχεια θα πρέπει να δημιουργήσουμε σαφείς κλάσεις παράγωγες της Shape για κάθε έναν επιμέρους τύπο γεωμετρικού σχήματος.

```
public class Circle extends Shape {

    private double r;
```

```
        public Circle(double r) {
            this.r = r;
        }

        @Override
        public double getArea() {
            return 2 * Math.PI * r;
        }

        @Override
        public String toString() {
            return "Κύκλος ακτίνας " + r;
        }
    }
```

Κώδικας 13.16 *Η παράγωγη κλάση Circle.*

Παραθέτουμε εδώ μόνο τις μεθόδους που είναι αναγκαίες για την εφαρμογή. Αναγνώστες, ρυθμιστές και οι μέθοδοι equals και hashCode παραλείπονται για απλοποίηση της παρουσίασης των εννοιών της κληρονομικότητας και του πολυμορφισμού. Η κλάση Circle είναι παράγωγη της Shape, διαθέτει έναν δημιουργό που δίνει τιμή στην ακτίνα του κύκλου, υλοποιεί την αφηρημένη μέθοδο getArea της Shape και επανορίζει τη μέθοδο toString.
Παρόμοια είναι και η υλοποίηση της κλάσης των ορθογώνιων.

```
public class Rectangle extends Shape {

    private final double length;
    private final double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double getArea() {
        return length * width;
    }

    @Override
    public String toString() {
        return "Ορθογώνιο μήκους " + length + ", πλάτους " + width;
    }

    public double getLength() {
        return length;
    }

    public double getWidth() {
        return width;
    }
}
```

Κώδικας 13.17 *Η παράγωγη κλάση Rectangle.*

Εδώ έχουμε προσθέσει και τους αναγνώστες καθώς όπως βλέπουμε αμέσως παρακάτω είναι χρήσιμοι (ο ένας από τους δύο) στην κλάση των τετραγώνων.
Η κλάση των τετραγώνων μπορεί εύκολα να οριστεί ως παράγωγη της κλάσης Rectangle.

```
    public class Square extends Rectangle {
```

```
        public Square(double length) {
            super(length, length);
        }

        @Override
        public String toString() {
            return "Τετράνωνο πλευράς " + getLength();
        }
    }
```

Κώδικας 13.18 *Η κλάση Square παράγωγη της Rectangle.*

Η κλάση των τετραγώνων είναι παράγωγη της Rectangle, επομένως είναι παράγωγη και της Shape. Ο δημιουργός της κλάσης καλεί τον δημιουργό της γονικής όπου περνάει μήκος ίσο με πλάτος, μια και ένα τετράγωνο δεν είναι παρά ένα ορθογώνιο παραλληλόγραμμο με ίσο μήκος και πλάτος. Η μέθοδος getArea δεν επανορίζεται εδώ καθώς η κληρονομούμενη από την Rectangle είναι ικανοποιητική.
Τέλος, ακολουθεί η κλάση των τριγώνων:

```
public class Triangle extends Shape {

    private final double base;
    private final double high;

    public Triangle(double base, double high) {
        this.base = base;
        this.high = high;
    }

    @Override
    public double getArea() {
        return base * high / 2;
    }

    @Override
    public String toString() {
        return "Τρίγωνο βάσης " + base + ", ύψους=" + high;
    }
}
```

Κώδικας 13.19 *Η παράγωγη κλάση Triangle.*

Τώρα είμαστε έτοιμοι για να αναπτύξουμε την εφαρμογή σύμφωνα με τις απαιτήσεις της άσκησης.

```
    import java.text.DecimalFormat;
    import java.text.DecimalFormatSymbols;
    import java.util.ArrayList;

    public class CalcAreas {

        public static void main(String[] args) {//Κώδικας 13.20
            ArrayList<Shape> shapes = new ArrayList<>();
            shapes.add(new Circle(4.5));
            shapes.add(new Circle(6));
            shapes.add(new Rectangle(3.8, 5));
            shapes.add(new Rectangle(6, 2.6));
            shapes.add(new Square(9));
            shapes.add(new Square(3));
            shapes.add(new Triangle(4.4, 2));
            shapes.add(new Triangle(4.4, 2.2));
            DecimalFormatSymbols dfs = new DecimalFormatSymbols();
```

```
            dfs.setDecimalSeparator('.');
            dfs.setGroupingSeparator(',');
            DecimalFormat f = new DecimalFormat("######.###");
            f.setDecimalFormatSymbols(dfs);

            for (Shape shape : shapes) {
                System.out.println(shape + ", εμβαδόν: " +
f.format(shape.getArea()));
            }
        }
}
```

Κώδικας 13.20 *Η εφαρμογή CalcAreas.*

Σύμφωνα με την εκφώνηση, η εφαρμογή μας φορτώνει οκτώ γεωμετρικά σχήματα σε ένα ArrayList και στη συνέχεια εμφανίζει την String αναπαράσταση και το εμβαδόν καθενός. Παρατηρήστε πως κατά τη διαμόρφωση της εξόδου καθορίζουμε το διαχωριστικό δεκαδικών να είναι ο χαρακτήρας '.' και χιλιάδων να είναι ο χαρακτήρας ','.

13.4.2 Η κλάση Quiz

Να υλοποιήσετε την κλάση Quiz που διαθέτει τις ακόλουθες λειτουργίες:

1. Παρέχει λειτουργία πρόσθεσης ερωτήσεων.
2. Υποστηρίζει ερωτήσεις σωστού-λάθους και πολλαπλών επιλογών
3. Κατά την εκτέλεσή της, υποβάλλει τον χρήστη σε τεστ με βάση τις καταχωρημένες ερωτήσεις.
4. Για κάθε ερώτηση επιβεβαιώνει τη σωστή απάντηση ή αν ο χρήστης απαντήσει λανθασμένα, εμφανίζει τη σωστή απάντηση.

Λύση

Από την εκφώνηση είναι φανερό πως έχουμε να διαχειριστούμε ερωτήσεις δύο τύπων, σωστού-λάθους και πολλαπλών επιλογών. Και οι δύο τύποι όμως, παρότι εξειδικεύονται, παραμένουν ερωτήσεις. Επομένως, ένας σχεδιασμός με γονική κλάση, αυτή των ερωτήσεων και παράγωγες τις κλάσεις των ερωτήσεων σωστού-λάθους και πολλαπλών επιλογών θα φανεί χρήσιμος.

Οι λειτουργίες που είναι αναγκαίο να δηλωθούν στη γονική κλάση είναι η λήψη του κειμένου της ερώτησης και η αξιολόγηση της απάντησης. Οι λειτουργίες αυτές μπορούν να υλοποιηθούν στις παράγωγες, όχι όμως και στη γονική. Ως εκ τούτου, η γονική θα είναι αφηρημένη.

```
public abstract class Question {
    public abstract String getQuestion();
    public abstract boolean evaluateAnswer(String answer);
}
```

Κώδικας 13.21 *Η αφηρημένη κλάση Question.*

Στη συνέχεια, στον κώδικα 13.22, παρουσιάζεται η υλοποίηση της κλάσης των ερωτήσεων σωστού-λάθους.

```
public class TrueFalse extends Question {
    String text;
    boolean correctAnswer;

    public TrueFalse(String text, boolean answer) {
        this.text = text;
        this.correctAnswer = answer;
    }


    @Override
    public String getQuestion() {
        return text +"\n t(rue) or f(alse)\n";
    }
```